

Praktikum 9

Thema: Generics

Aufgabe 1: Klassenschabole (20 Minuten)

Die folgenden drei Klassen machen exakt das Gleiche – nur mit unterschiedlichen Datentypen:

```
class IntBox {
    private int value;

    public IntBox(int value) {
        this.value = value;
    }
    public int getValue() {
        return value;
    }
}

class StringBox {
    private String value;

    public StringBox(String value) {
        this.value = value;
    }
    public String getValue() {
        return value;
    }
}

class DoubleBox {
    private double value;

    public DoubleBox(double value) {
        this.value = value;
    }
    public double getValue() {
        return value;
    }
}
```

Das ist natürlich unschön... Die Hauptklasse sieht so aus:

```
public class Main {  
    public static void main(String[] args) {  
        IntBox ib = new IntBox(10);  
        StringBox sb = new StringBox("Hallo");  
        DoubleBox db = new DoubleBox(3.14);  
  
        System.out.println(ib.getValue());  
        System.out.println(sb.getValue());  
        System.out.println(db.getValue());  
    }  
}
```

Ersetzen Sie die drei Klassen `IntBox`, `StringBox` und `DoubleBox` durch **eine einzige generische Klasse** und passen Sie anschließend die `main`-Methode entsprechend an.

Aufgabe 2: Methodenschablone (20 Minuten)

Schreiben Sie ein Java-Programm, das eine statische generische **Methode** `min` verwendet, die zwei Zahlen bekommt und die kleinere der beiden Zahlen zurückgibt.

Die Methode soll für verschiedene numerische Typen funktionieren, z. B.:

- `Integer`
- `Float`
- `Double`

Sie sollten dafür eine Methodenschablone mit Einschränkung auf die Klasse `Number` verwenden.

Hinweis: Da `Number` keine Vergleichsmethode besitzt, sollten Sie die Werte über die Methode `doubleValue()` aus `Number` vergleichen, die für beliebige Zahlen eben den `double`-Wert liefert. Zeigen Sie in der `main`-Methode, wie die `min`-Methode für `Integer`-, `Float`- und `Double`-Werte funktioniert!

Aufgabe 3: Arrays (20 Minuten)

Schreiben Sie ein Java-Programm mit einer statischen generischen Methode `average`, die den Durchschnitt eines Zahlen-Arrays berechnet. Es sollen Arrays mit beliebigen Unter-Datentypen von `Number` möglich sein.

Die passende `main`-Methode dazu sei vorgegeben:

```
public class Main {  
    public static void main(String[] args) {  
        Integer[] ints = {1, 2, 3, 4};  
        Double[] doubles = {2.0, 4.0, 6.0};  
  
        System.out.println(average(ints));  
        System.out.println(average(doubles));  
    }  
}
```

Bringen Sie also das Programm zum Laufen!

Aufgabe 4: Zusatzaufgabe (30 Minuten)

Gegeben sei das folgende Interface:

```
public interface Task {  
    String getTitle();  
    int getPriority();  
}
```

Implementieren Sie zwei Klassen:

- `SimpleTask`: Diese Klasse setzt das Interface um und hat zusätzlich einen Konstruktor für die zwei Attribute `String title` und `int priority`.
- `ComplexTask`: Diese Klasse setzt ebenfalls das Interface um (leitet sich aber nicht von `SimpleTask` ab!), und hat zusätzlich ein Attribut für `int personDays`, in der der Aufwand in Personentagen für die Aufgabe gespeichert sind. Der Konstruktor braucht also drei Argumente und die Klasse braucht eine zusätzliche Methode `getPersonDays()`.

Die Hauptklasse für diese Aufgabe sieht so aus:

```
package aufgabe4;

public class Aufgabe4 {

    public static void main(String [] args) {

        TaskPlan<Task> plan = new TaskPlan<>();

        plan.add(new SimpleTask("Login fixen", 3));
        plan.add(new SimpleTask("UI verbessern", 2));
        plan.add(new ComplexTask("Bug report schreiben", 1, 15));

        System.out.println("Es gibt " + plan.size() + " Aufgaben");

        Task found1 = plan.findByTitle("UI verbessern");
        System.out.println("Prio von UI verbessern: " +
                           found1.getPriority());

        Task found2 = plan.findByTitle("Bug report schreiben");
        System.out.println("Prio von Bug report schreiben: " +
                           found2.getPriority());

    }

}
```

Bringen Sie das Programm zum Laufen, indem Sie die noch fehlende Klasse `TaskPlan` implementieren. Die Klasse kann:

1. beliebige Tasks aufnehmen (mit der `add`-Methode)
2. sie kennt die Gesamtanzahl der Tasks (`size`-Methode) und
3. kann für eine bestimmte Task die Priorität bestimmen (`getPriority`-Methode).

Achtung: Diese Klasse sollte generisch für alle Typen von Tasks funktionieren!